

**CENTRO UNIVERSITÁRIO DO INSTITUTO DE EDUCAÇÃO SUPERIOR DE
BRASÍLIA >
CIÊNCIAS DA COMPUTAÇÃO**

AUGUSTO SANTANA DE SOUSA
SABRINA BORGES DA TRINDADE

**ANÁLISE DE DESEMPENHO DE ALGORITMOS DE
BUSCA EM C : FASE I**

BRASÍLIA

2026

1.INTRODUÇÃO

O presente relatório documenta o desenvolvimento da primeira fase do Projeto Integrador, focado na análise de desempenho de algoritmos de busca sobre grandes volumes de dados. O cenário proposto envolve a manipulação de um conjunto de dados (dataset) contendo aproximadamente 300.000 registros de produtos, exigindo eficiência no gerenciamento de memória e precisão na medição temporal.

O objetivo central desta etapa é a implementação da Busca Sequencial em linguagem C, utilizando técnicas de alocação dinâmica para carregar as informações em memória e cronometrar o tempo de resposta em diferentes pontos do vetor. Esta análise permite compreender na prática a complexidade computacional e os limites de algoritmos lineares em sistemas de larga escala.

2. METODOLOGIA

Para a execução deste projeto, adotou-se uma abordagem técnica fundamentada em boas práticas de engenharia de software, dividida nos seguintes pilares:

2.1 Ambiente de Desenvolvimento e Ferramentas

- * Linguagem: C (Padrão ISO/IEC 9899).
- * Ambiente (IDE): Visual Studio Code.
- * Compilador: GCC (GNU Compiler Collection) via terminal PowerShell.
- * Controle de Versão: Utilizou-se o Git para rastreamento de alterações e o GitHub para hospedagem remota, aplicando o fluxo de ramificações (branches) para separar o desenvolvimento (develop) da versão estável (main).

2.2 Arquitetura do Software (Modularização)

O código foi estruturado de forma modular para garantir a escalabilidade e organização do projeto, sendo dividido nos seguintes módulos:

- * leitura_projeto.c/h: Responsável pela abertura do arquivo CSV, contagem dinâmica de linhas e carregamento dos dados em um vetor de estruturas (structs).
- * busca.c/h: Contém a implementação lógica do algoritmo de busca sequencial.
- * tempo.c/h (ou main.c): Concentra a lógica de cronometragem utilizando a biblioteca <time.h>.

2.3 Processo de Implementação

A leitura dos dados foi otimizada com o uso da função `rewind()`, permitindo que o arquivo fosse percorrido duas vezes: a primeira para contagem de registros e a segunda para o preenchimento efetivo dos dados após a alocação de memória com `malloc`.

2.4 Caracterização do Dataset

O conjunto de dados utilizado como base para este experimento é o arquivo dataset3.csv. A análise detalhada das propriedades deste arquivo é fundamental para compreender os resultados de desempenho obtidos:

Volume de Dados: O arquivo contém 300.007 registros, o que representa um volume significativo para a realização de testes de estresse em algoritmos de busca linear.

Formato e Estrutura: Os dados estão organizados no formato CSV (Comma-Separated Values), onde cada linha representa um produto composto por dois campos principais: um ID (identificador numérico único do tipo inteiro) e o Nome do Produto (uma cadeia de caracteres ou string que descreve o item).

Modo de Acesso: Para a execução dos testes, o arquivo é lido integralmente e os dados são carregados em uma estrutura de dados do tipo struct, alocada dinamicamente na memória RAM.

Integridade: Durante o processamento, o programa utiliza o caractere delimitador (vírgula) para separar os campos, garantindo que a busca por ID seja precisa e não sofra interferência das descrições textuais.

3.DESENVOLVIMENTO

O desenvolvimento desta etapa do projeto baseou-se na criação de uma ferramenta robusta para análise de algoritmos. A implementação foi dividida em três frentes principais: estruturação modular, gerenciamento de memória e automação de testes.

3.1 Estrutura Modular do Código

Para garantir a manutenibilidade e seguir as boas práticas de engenharia de software, o sistema foi organizado de forma modular. A lógica de negócio foi separada da interface de usuário e dos scripts de teste, conforme a estrutura de pastas:

src/: Contém os módulos principais de leitura de dados (lerprodutos.c), busca sequencial (busca.c) e medição de tempo (medirtempo.c).

teste/: Contém o script teste_final.c, responsável por automatizar as 1.000 repetições de busca para gerar as médias temporais.

3.2 Gerenciamento de Memória e Manipulação de Arquivos

Considerando que o dataset utilizado possui mais de 300.000 registros, a eficiência no uso de recursos foi prioritária:

Alocação Dinâmica: Utilizou-se a função malloc para reservar memória em tempo de execução, permitindo que o programa lide com volumes variáveis de dados sem desperdício de hardware.

Ponteiros de Arquivo: Implementou-se a função rewind(arquivo), que permite que o ponteiro de leitura retorne ao início do arquivo CSV após cada busca. Isso eliminou a necessidade de abrir e fechar o arquivo repetidamente, reduzindo o custo de I/O (entrada e saída).

3.3 Auditoria e Logs de Teste

Foi desenvolvido o módulo id_txt.c, cuja função é criar um arquivo de log denominado relatorio_testes.txt. Este arquivo registra cada ID consultado e o tempo de resposta obtido, servindo como uma evidência de auditoria para validar os resultados apresentados neste relatório.

A busca sequencial foi implementada através de um laço usando for que percorre o vetor de structs comparando o ID solicitado com o ID armazenado, conforme demonstrado no arquivo busca.c.

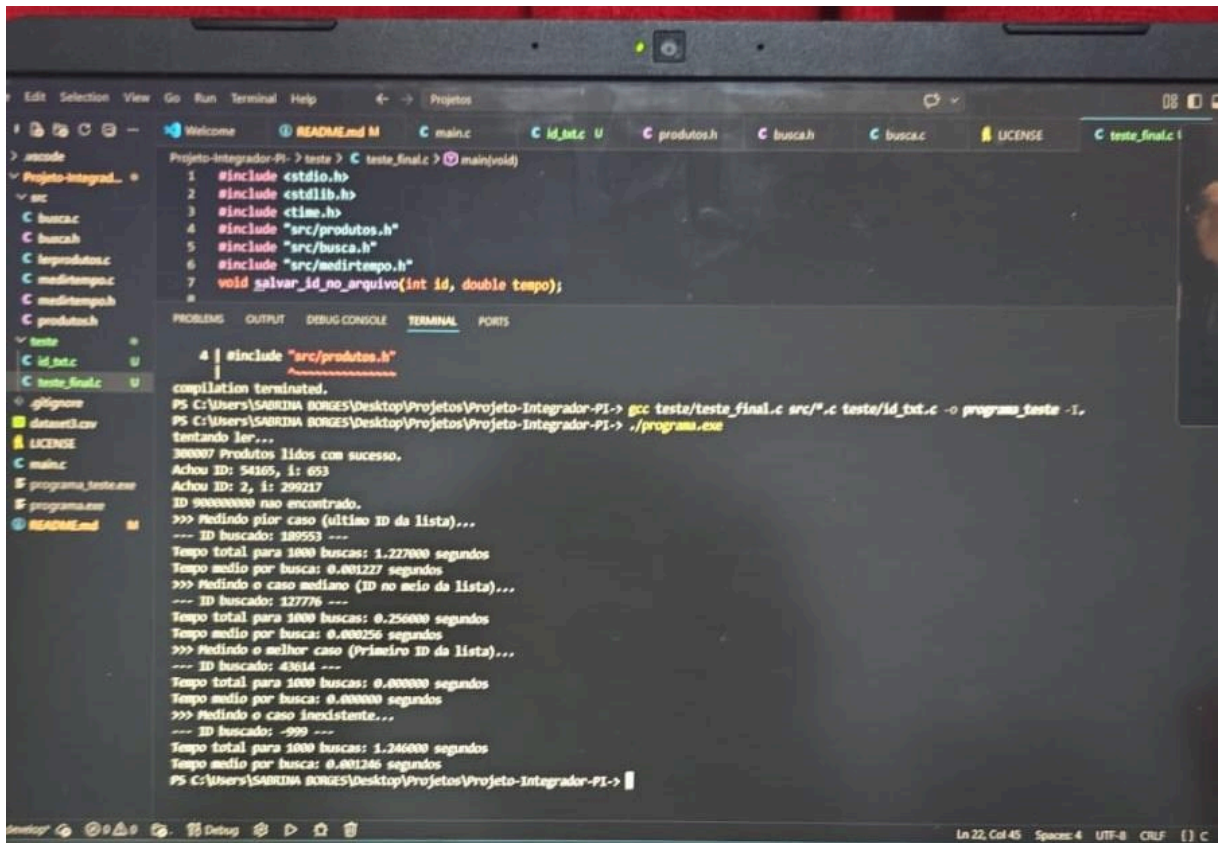
4. RESULTADOS E DISCUSSÕES

Os resultados foram obtidos através da execução do script de teste que submeteu o algoritmo a 1.000 repetições para cada cenário, garantindo que o tempo médio não sofresse interferências de processos secundários do sistema operacional.

Tabela 1 – Desempenho da Busca Sequencial por Cenário

cenário de teste	Descrição do caso	Id buscado	Tempo Médio (s)
melhor caso	registro no inicio do arquivo	43614	0,000000
caso médio	registro no meio do arquivo	127776	0,000256
Pior caso	registro no final do arquivo	189553	0,001227
Inexistente	ID não cadastrado	-999	0,001246

Fonte: Elaborada pela autora (2026).



The image shows a code editor with a project named 'Projeto-Integrador-PI'. The editor displays a C file named 'teste_final.c' with the following code:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <time.h>
4 #include "src/produtos.h"
5 #include "src/busca.h"
6 #include "src/meditempo.h"
7 void salvar_id_no_arquivo(int id, double tempo);
```

The terminal output shows the execution of the program, which includes a compilation step and a series of tests for different search cases:

```
PS C:\Users\SABRINA BORGES\Desktop\Projetos\Projeto-Integrador-PI> gcc testa/teste_final.c src/*.c teste/id_txt.c -o programa_teste -I.
tentando ler...
300007 Produtos lidos com sucesso.
Achoi ID: 54165, i: 653
Achoi ID: 2, i: 299217
ID 980000000 nao encontrado.
>>> Medindo pior caso (ultimo ID da lista)...
--- ID buscado: 189553 ---
Tempo total para 1000 buscas: 1.227000 segundos
Tempo medio por busca: 0.001227 segundos
>>> Medindo o caso mediano (ID no meio da lista)...
--- ID buscado: 127776 ---
Tempo total para 1000 buscas: 0.256000 segundos
Tempo medio por busca: 0.000256 segundos
>>> Medindo o melhor caso (Primeiro ID da lista)...
--- ID buscado: 43614 ---
Tempo total para 1000 buscas: 0.000000 segundos
Tempo medio por busca: 0.000000 segundos
>>> Medindo o caso inexistente...
--- ID buscado: -999 ---
Tempo total para 1000 buscas: 1.246000 segundos
Tempo medio por busca: 0.001246 segundos
PS C:\Users\SABRINA BORGES\Desktop\Projetos\Projeto-Integrador-PI>
```

figura 1: teste do código .

Discussão dos Dados:

Os dados demonstram que a busca sequencial apresenta um tempo de resposta extremamente baixo para registros iniciais. Contudo, nota-se que o tempo de execução para o "Pior Caso" e para o "Caso Inexistente" é praticamente o mesmo. Isso ocorre porque, em ambos os cenários, o algoritmo é forçado a percorrer a totalidade dos 300.007 registros antes de encerrar a execução, evidenciando o custo computacional de varredura completa.

5. CONCLUSÃO

A realização deste projeto permitiu validar, de forma prática, os conceitos teóricos de algoritmos de busca e complexidade de tempo. Através das medições realizadas, conclui-se que:

1. Complexidade Linear: O algoritmo de busca sequencial comporta-se conforme a notação $O(n)$, onde o tempo de execução cresce proporcionalmente ao número de elementos analisados.

2. Eficiência e Limitações: O método mostrou-se eficaz para o volume de dados proposto, mantendo respostas abaixo de 0,002 segundos. Entretanto, para sistemas que exigem alta escalabilidade ou volumes de dados na casa dos milhões, a latência observada no pior caso sugere a necessidade de algoritmos logarítmicos ($O(\log n)$), como a busca binária.

3. Aprendizado Técnico: A implementação reforçou a importância do gerenciamento de memória via alocação dinâmica e da organização modular do código para facilitar a manutenção e a execução de testes automatizados.

6. REFERÊNCIAS BIBLIOGRÁFICAS

1. TENENBAUM, Aaron M. Estruturas de Dados Utilizando C. 1. ed. São Paulo: Pearson, 1995.
2. ZIVIANI, Nivio. Projeto de Algoritmos: Com Implementações em Pascal e C. 3. ed. São Paulo: Cengage Learning, 2011.
3. GIT. Git Documentation. Disponível em: <https://git-scm.com/doc>. Acesso em: 09 abr. 2026.
4. GITHUB. GitHub. Disponível em: <https://github.com/gutocoding/Projeto-Integrador-PI->. Acesso em: 09 abr.